# Parrot API-to-Backend Mapping

Conor Hoekstra | NVIDIA

Table 1: Parrot method mapping (core transforms, masking, and reductions).

| Parrot (C++) | Parrot Python | CUDA C++ backend | cuda.compute backend |
|---|---|---|---|
| map(op) | map(op) | thrust::make_transform_iterator | iterators.TransformIterator |
| map2(arr, op) | map2(arr, op) | thrust::make_zip_iterator + thrust::make_transform_iterator | iterators.ZipIterator + iterators.TransformIterator |
| map2(scalar, op) | map2(scalar, op) | thrust::make_transform_iterator | clone + map(bound_op) |
| times(arg) | times(arg) | 🦜 map2(arg,mul) | 🦜 map2(arg,mul_op) |
| add(arg) | add(arg) | 🦜 map2(arg,add) | 🦜 map2(arg,add_op) |
| minus(arg) | minus(arg) | 🦜 map2(arg,minus) | 🦜 map2(arg,sub_op) |
| min(arg) | min(arg) | 🦜 map2(arg,min) | 🦜 map2(arg,min_op) |
| max(arg) | max(arg) | 🦜 map2(arg,max) | 🦜 map2(arg,max_op) |
| div(arg) | div(arg) | 🦜 map2(arg,div) | 🦜 map2(arg,div_op) |
| mod(arg) | mod(arg) | 🦜 map2(arg,mod) | 🦜 map2(arg,mod_op) |
| idiv(arg) | idiv(arg) | 🦜 map2(arg,idiv) | 🦜 map2(arg,idiv_op) |
| as<T> | astype(dtype) | 🦜 map(cast_functor) | 🦜 map(cast_op) |
| gte(arg) | gte(arg) | 🦜 map2(arg,gte) | 🦜 map2(arg,gte_op) |
| lte(arg) | lte(arg) | 🦜 map2(arg,lte) | 🦜 map2(arg,lte_op) |
| gt(arg) | gt(arg) | 🦜 map2(arg,gt) | 🦜 map2(arg,gt_op) |
| lt(arg) | lt(arg) | 🦜 map2(arg,lt) | 🦜 map2(arg,lt_op) |
| eq(arg) | eq(arg) | 🦜 map2(arg,eq) | 🦜 map2(arg,eq_op) |
| neq(arg) | neq(arg) | 🦜 map2(arg,neq) | 🦜 map2(arg,neq_op) |
| dble() | double() | 🦜 map(double_functor) | 🦜 map(double_op) |
| half() | half() | 🦜 map(half_functor) | 🦜 map(half_op) |
| abs() | abs() | 🦜 map(abs_functor) | 🦜 map(abs_op) |
| log() | log() | 🦜 map(log_functor) | 🦜 map(log_op) |
| exp() | exp() | 🦜 map(exp_functor) | 🦜 map(exp_op) |
| neg() | neg() | 🦜 map(negate) | 🦜 map(neg_op) |
| odd() | odd() | 🦜 map(odd_functor) | 🦜 map(odd_op) |
| even() | even() | 🦜 map(even_functor) | 🦜 map(even_op) |
| sign() | sign() | 🦜 map(sign_functor) | 🦜 map(sign_op) |
| sqrt() | sqrt() | 🦜 map(sqrt_functor) | 🦜 map(sqrt_op) |
| sq() | sq() | 🦜 map(sq_functor) | 🦜 map(square_op) |
| rand() | rand() | zip(index,value) + transform random functor | ZipIterator + TransformIterator random op |
| map_adj(op) | map_adj(op) | thrust::make_zip_iterator + thrust::make_transform_iterator (zip_function) | PermutationIterator + ZipIterator + TransformIterator |
| differ() | differ() | 🦜 map_adj(neq) | 🦜 map_adj(neq_op) |
| deltas() | deltas() | 🦜 map_adj(delta) | 🦜 map_adj(delta_op) |
| keep(mask) | keep(mask) | mask-aware fusion_array constructor | lazy mask in ParrotArray (_mask) |
| filter(pred) | filter(predicate) | thrust::make_transform_iterator + 🦜 keep | algorithms.select |
| where() | where() | 🦜 range(size).keep(this) | 🦜 range(length).keep(self) |
| gather(indices) | gather(indices) | thrust::make_permutation_iterator | iterators.PermutationIterator |
| reduce<0>(init, op) | reduce(op, init, axis=0) | thrust::reduce | algorithms.reduce_into |
| reduce<1>(init, op) | reduce(op, init, axis=1) | 🦜 transpose().reduce<2>() | algorithms.segmented_reduce |
| reduce<2>(init, op) | reduce(op, init, axis=2) | thrustx::reduce_by_n | algorithms.segmented_reduce |
| sum<axis>() | sum(axis) | 🦜 reduce<axis>(0,+) | 🦜 reduce(+,0,axis) |
| prod<axis>() | prod(axis) | 🦜 reduce<axis>(1,*) | 🦜 reduce(*,1,axis) |
| maxr<axis>() | maxr/max | 🦜 reduce<axis>(init,max) | 🦜 reduce(max,init,axis) |
| minr<axis>() | minr/min | 🦜 reduce<axis>(init,min) | 🦜 reduce(min,init,axis) |
| any<axis>() | any(axis) | 🦜 reduce<axis>(init,logical_or) | 🦜 reduce(_reduce_or,...) |
| all<axis>() | all(axis) | 🦜 reduce<axis>(init,logical_and) | 🦜 reduce(_reduce_and,...) |

Table 2: Parrot method mapping (scans, shape/rank operations, and products).

| Parrot (C++) | Parrot Python | CUDA C++ backend | cuda.compute backend |
|---|---|---|---|
| scan<0>(op) | scan(op, axis=0) | thrust::inclusive_scan | algorithms.inclusive_scan |
| scan<1>(op) | scan(op, axis=1) | 🦜 transpose().scan<2>().transpose() | ⛔ (need scan_by_key) |
| scan<2>(op) | scan(op, axis=2) | thrust::inclusive_scan_by_key | ⛔ (need scan_by_key) |
| sums<axis>() | sums(axis) | 🦜 scan<axis>(+) | 🦜 _scan_to_array(+, 0) |
| prods<axis>() | prods(axis) | 🦜 scan<axis>(*) | 🦜 _scan_to_array(*, 1) |
| mins<axis>() | mins(axis) | 🦜 scan<axis>(min) | 🦜 _scan_to_array(min, init) |
| maxs<axis>() | maxs(axis) | 🦜 scan<axis>(max) | 🦜 _scan_to_array(max, init) |
| alls<axis>() | alls(axis) | 🦜 scan<axis>(logical_and) | 🦜 scan(logical_and,...) |
| anys<axis>() | anys(axis) | 🦜 scan<axis>(logical_or) | 🦜 scan(logical_or,...) |
| reshape(shape) | reshape(shape) | shape metadata + iterator view truncation | shape metadata update (lazy length/shape change) |
| cycle(shape) | cycle(shape) | thrustx::make_cycle_iterator | ⛔ (no direct cycle API in current Python file) |
| repeat(n) (scalar) | repeat(n) | thrust::make_constant_iterator | 🦜 constant(front(),n) |
| replicate(int n) | replicate(n) | thrustx::make_replicate_iterator | TransformIterator(idx//n) + PermutationIterator |
| replicate(mask) | ⛔ (mask-overload) | materialized scatter + inclusive scan + permutation copy | ⛔ |
| cross(other) | cross(other) | 🦜 replicate + cycle + pairs | 🦜 replicate + gather(mod) + ZipIterator |
| outer(other, op) | outer(other, op) | thrustx::make_outer_iterator | Transform/Permutation/Zip iterator composition |
| pairs(other) | pairs(other) | zip + pair transform iterator | iterators.ZipIterator |
| enumerate() | enumerate() | 🦜 pairs(range(size)) | ZipIterator(values, CountingIterator) |
| transpose() | transpose() | counting + transpose-index transform + permutation iterator | TransformIterator(idx) + PermutationIterator |
| rev() | rev() | thrust::make_reverse_iterator | iterators.ReverseIterator (with collect in some paths) |
| append(v) | append(v) | thrustx::make_append_iterator | index transform + PermutationIterator + TransformIterator |
| prepend(v) | prepend(v) | thrustx::make_prepend_iterator | index transform + PermutationIterator + TransformIterator |